

TITLE AND ABSTRACT

AI-WEATHERWISE API

AI-Based Weather Information and Recommendation API

TEAM INFORMATION

Project Title: AI-WeatherWise API

TEAM ID: 6ab6129afa5e5219dc386452

TEAM LEADER: VIDHYA D

MAIL ID: kumarpvk12@gmail.com

ROLE: PROJECT LEAD AND DOCUMENTATION

TEAM MEMBER: NISHA S

MAIL ID: nishasekar811@gmail.com

ROLE: FRONTEND DEVELOPER

TEAM MEMBER: AKSHAYA A

MAIL ID : akshayaarumugam9965@gmail.com

ROLE: BACKEND DEVELOPER

TEAM MEMBER: AKSHITHA T ;

MAIL ID: akshithat341@gmail.com

ROLE: DEMO VIDEO

TEAM MEMBER: AMBIKA P ;

MAIL ID: ambikaambika4545@gmail.com

ROLE: DOCUMENT CREATER

PROJECT DOCUMENTATION :

Project Type: Web Application and REST API

Programming Language: Python

Framework: Flask

Frontend: HTML and CSS

Data Format: JSON

Repository: GitHub

ABSTRACT

AI-WeatherWise API is a web-based weather information and recommendation system developed using Python and Flask. The main purpose of this project is to provide weather information to users in a simple and understandable format.

The application accepts a city name from the user and processes weather information. The system can display important weather parameters such as temperature, humidity, wind speed, and weather condition. In addition to displaying weather information, the project includes a recommendation component that provides simple advice based on the weather condition.

The project follows a modular architecture. The main Flask application handles requests, while separate service modules are used for weather processing and AI-based recommendations. HTML and CSS are used to create the user interface, and JSON is used for structured data communication.

When a live weather API is unavailable, mock data can be used to demonstrate the application workflow. Such data is treated only as demonstration data and is not presented as real-time weather information.

The project demonstrates practical concepts of Python programming, Flask development, API communication, JSON processing, frontend development, modular programming, and GitHub project management.

INTRODUCTION

1. INTRODUCTION

Weather information is important in everyday life. People check weather conditions before travelling, attending outdoor events, exercising, working, and planning daily activities. Modern websites and mobile applications provide weather information through APIs.

An Application Programming Interface, commonly known as an API, allows different software systems to communicate with each other. A weather API can provide information such as temperature, humidity, wind speed, rainfall, pressure, and weather conditions.

AI-WeatherWise API is developed to demonstrate how weather information can be processed and presented through a web application. The user enters a city name, and the Flask backend processes the request.

One of the important features of this project is the recommendation component. Instead of displaying only numerical values, the application can provide simple advice based on weather conditions.

For example, when the temperature is high, the application may recommend staying hydrated. When rainy conditions are detected, the application may recommend carrying an umbrella or planning outdoor activities carefully.

Flask is used as the backend framework because it is lightweight and suitable for developing web applications and APIs. HTML and CSS are used for the frontend interface.

The project is designed so that a real weather API can be integrated in the future. Advanced AI models, forecasting, location services, alerts, and mobile applications can also be added.

PROBLEM STATEMENT AND OBJECTIVES

2. PROBLEM STATEMENT

Traditional weather applications mainly display weather measurements such as temperature, humidity, and wind speed. Although these values are useful, users may still need a simple explanation of what the weather information means for their daily activities.

For example, a user may see a high temperature but may not know what precautions should be taken. Similarly, rainy weather may require different planning for outdoor activities.

Therefore, this project focuses on developing a simple system that combines weather information with understandable recommendations.

2.1 OBJECTIVES

The main objectives of AI-WeatherWise API are:

1. To develop a web-based weather application.
2. To use Python for backend development.
3. To use Flask for API and web application development.
4. To accept city input from the user.
5. To process weather information.
6. To display weather details clearly.
7. To generate simple weather-based recommendations.
8. To provide structured JSON responses.
9. To maintain the project using GitHub.
10. To demonstrate the application using mock data when a live API is unavailable.

2.2 SCOPE OF THE PROJECT

The project focuses on weather information processing and recommendation generation. It provides a basic foundation that can later be extended with real-time weather data, weather forecasting, location detection, notifications, advanced AI models, and mobile applications.

The project is mainly designed for educational and demonstration purposes.

SYSTEM REQUIREMENTS

3. SYSTEM REQUIREMENTS

System requirements define the hardware and software needed to develop and execute the AI-WeatherWise API.

3.1 HARDWARE REQUIREMENTS

The project can be developed using a basic laptop or desktop computer.

The recommended hardware includes:

- Processor: Dual-core or better
- RAM: Minimum 4 GB
- Storage: Minimum 1 GB free space
- Keyboard and mouse
- Display
- Internet connection

No high-end hardware is required for the basic project.

3.2 SOFTWARE REQUIREMENTS

The software requirements are:

- Operating System: Windows, Linux, or macOS
- Programming Language: Python
- Framework: Flask
- Frontend: HTML5 and CSS3
- Data Format: JSON
- Code Editor: Visual Studio Code or equivalent
- Browser: Chrome, Edge, Firefox, or equivalent
- Version Control: Git
- Repository: GitHub

3.3 FUNCTIONAL REQUIREMENTS

The system should allow the user to enter a city name. It should process the request and display weather information and an appropriate recommendation.

The system should also be able to provide structured JSON data.

3.4 NON-FUNCTIONAL REQUIREMENTS

The system should be simple, user-friendly, maintainable, modular, and responsive.

The application should provide understandable error messages. Sensitive API keys should be protected and should not be uploaded to a public GitHub repository.

SYSTEM ARCHITECTURE AND WORKFLOW

4. SYSTEM ARCHITECTURE

The AI-WeatherWise API follows a modular architecture consisting of the following components:

4.1 USER INTERFACE

The HTML and CSS interface allows the user to enter a city name and view the weather information.

4.2 FLASK APPLICATION

The Flask application receives the user's request, validates the input, communicates with the required services, and prepares the final response.

4.3 WEATHER SERVICE

The weather service is responsible for obtaining and processing weather information. In a real implementation, it can communicate with an external weather API.

4.4 AI SERVICE

The AI service analyzes weather information and generates a simple recommendation.

4.5 RESPONSE

The processed information is returned to the user interface and displayed on the dashboard.

4.6 SYSTEM WORKFLOW

The complete workflow is:

User → Web Interface → Flask Backend → Weather Service → Weather Data → AI Service → Recommendation → Final Response → User

The modular design makes the application easier to maintain and extend.

A real weather API can be connected later without changing the complete project structure.

TECHNOLOGIES AND PROJECT STRUCTURE

5. TECHNOLOGIES USED

5.1 PYTHON

Python is used as the main programming language. It provides simple syntax and libraries for web development and artificial intelligence.

5.2 FLASK

Flask is a lightweight Python web framework used for creating routes, handling requests, and generating responses.

5.3 HTML

HTML is used to create the structure of the webpage, including headings, forms, buttons, and weather result sections.

5.4 CSS

CSS is used to control the appearance of the application, including layout, spacing, fonts, borders, and buttons.

5.5 JSON

JSON is used for structured communication between the backend and other applications.

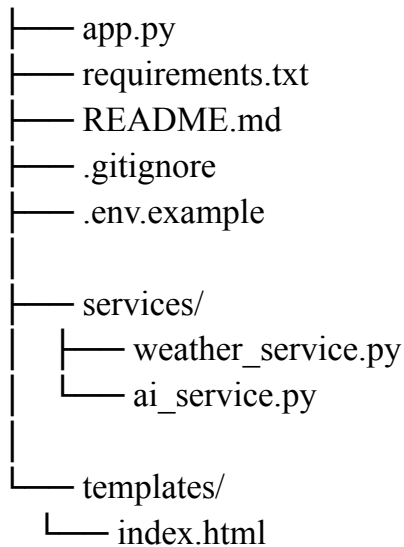
5.6 GITHUB

GitHub is used for storing and sharing the project source code and documentation.

5.7 PROJECT STRUCTURE

AI-WeatherWise-API/

|



The `app.py` file contains the main Flask application.

The `weather_service.py` file manages weather-related processing.

The `ai_service.py` file generates recommendations.

The `index.html` file provides the user interface.

The `requirements.txt` file contains the required Python packages.

API DESIGN AND IMPLEMENTATION

6. API DESIGN

An API allows software components to communicate with each other. In AI-WeatherWise API, the user provides a city name, and the backend processes the request.

6.1 API INPUT

The main input is the city name.

Example:

City: Chennai

The Flask application receives this value and sends it to the weather service.

6.2 API RESPONSE

The response can contain:

- City
- Temperature
- Humidity
- Wind Speed
- Weather Condition
- AI Advice

6.3 SAMPLE JSON RESPONSE

```
{  
  "city": "Chennai",  
  "temperature": 29,  
  "humidity": 78,  
  "condition": "Partly Cloudy",  
  "wind_speed": 14,  
  "ai_advice": "Stay hydrated and plan outdoor activities according to current weather conditions."  
}
```

The above values are demonstration values when mock data is being used.

6.4 IMPLEMENTATION

The Flask application receives the city name from the user interface. After validating the input, the application calls the weather service.

The weather service can retrieve live information from an external provider when a valid API key is available.

The returned information is passed to the AI service.

The AI service generates a recommendation based on the weather conditions.

The final result is returned to the HTML page.

6.5 API KEY SECURITY

A real weather API normally requires a valid API key. The key should be stored using an environment variable.

A placeholder can be used in project documentation:

YOUR_API_KEY_HERE

A real API key should not be uploaded to a public GitHub repository.

USER INTERFACE AND DEMONSTRATION

7. USER INTERFACE

The AI-WeatherWise interface is designed to be simple and user-friendly.

The main dashboard contains:

- Project title
- City input field
- Get Weather button
- Weather information section
- AI recommendation section

The user enters a city name and clicks the Get Weather button.

The backend processes the request and returns the weather information.

7.1 DEMONSTRATION EXAMPLE

For demonstration purposes, Chennai can be used as the selected city.

Example output:

City: Chennai

Temperature: 29°C

Condition: Partly Cloudy

Humidity: 78%

Wind Speed: 14 km/h

AI Weather Advice:

"Stay hydrated and plan outdoor activities according to current weather conditions."

These values are demonstration values and should not be treated as current real-time weather.

7.2 DEMONSTRATION WORKFLOW

The demonstration follows these steps:

1. Open the WeatherWise application.
2. Enter the city name.
3. Click the Get Weather button.
4. Flask receives the request.
5. Weather service processes the data.
6. AI service generates the recommendation.
7. Weather information is displayed.
8. AI advice is displayed.
9. JSON response can be viewed.

7.3 DEMONSTRATION VIDEO

The project demonstration video presents the application title, dashboard, sample weather information, AI recommendation, JSON response, and completion screen.

The video can be uploaded to the GitHub repository along with the source code.

TESTING AND RESULTS

8. TESTING

Testing is performed to verify whether the major functions of the application work correctly.

TEST CASE 1 – VALID CITY

Input: Chennai

Expected Result: Weather information and recommendation should be displayed.

Result: Successful for demonstration workflow.

TEST CASE 2 – EMPTY INPUT

Input: Empty city field.

Expected Result: The application should request a city name.

TEST CASE 3 – MOCK WEATHER DATA

Input: Configured demonstration city.

Expected Result: Predefined weather information should be displayed.

TEST CASE 4 – JSON RESPONSE

Input: Weather request.

Expected Result: Structured weather information should be returned in JSON format.

TEST CASE 5 – API FAILURE

Input: Invalid or unavailable external weather service.

Expected Result: The application should display an understandable error message or use clearly identified demonstration data.

8.1 RESULTS

The project successfully demonstrates the workflow of a weather information and recommendation application.

The system accepts user input, processes weather information, generates a recommendation, and displays the result through the dashboard.

The modular architecture makes the project easier to maintain and extend.

The mock-data mode allows the project to be demonstrated even when a live API key is not available.

The project also demonstrates how GitHub can be used to store the source code, documentation, and demonstration video.

8.2 ADVANTAGES

1. Simple user interface.
2. Lightweight Flask backend.
3. Modular architecture.
4. Structured JSON response.
5. Recommendation feature.
6. Easy GitHub sharing.
7. Mock-data support.

8. Easy future API integration.

8.3 LIMITATIONS

The current demonstration may use mock weather data when a valid external API key is unavailable. Therefore, mock values do not represent real-time weather information.

The recommendation system may also use simple rules instead of an advanced AI model.

FUTURE ENHANCEMENT, CONCLUSION AND REFERENCES

9. FUTURE ENHANCEMENTS

The project can be improved with several additional features.

REAL-TIME WEATHER

A valid weather API can be connected to provide current weather information.

WEATHER FORECAST

The application can provide five-day or seven-day weather forecasts.

LOCATION DETECTION

GPS or location services can automatically identify the user's location.

ADVANCED AI

An advanced AI model can generate personalized recommendations based on multiple weather parameters.

WEATHER ALERTS

The system can provide alerts for extreme temperatures, heavy rain, or strong winds.

MULTILINGUAL SUPPORT

The application can provide weather recommendations in multiple languages.

MOBILE APPLICATION

The API can be connected to an Android or iOS application.

CLOUD DEPLOYMENT

The Flask application can be deployed to a cloud platform and accessed through the internet.

10. CONCLUSION

AI-WeatherWise API demonstrates the development of a web-based weather information and recommendation system using Python, Flask, HTML, CSS, and JSON.

The application combines weather information with simple recommendations to make weather information easier to understand.

The modular architecture separates the main Flask application, weather service, AI service, and frontend. This makes the application easier to maintain and extend.

The project also demonstrates the use of mock data for application demonstration when a live weather API key is unavailable. Mock values are clearly identified as demonstration data.

Through this project, practical knowledge of Flask development, API communication, JSON processing, frontend development, modular programming, error handling, API-key security, testing, and GitHub project management is gained.

10.1 REFERENCES

1. Python Documentation – Python programming concepts and libraries.
2. Flask Documentation – Flask framework and web application development.
3. HTML5 Documentation – Webpage structure and forms.
4. CSS Documentation – Web design and styling.
5. JSON Documentation – Structured data communication.
6. Git Documentation – Version control.
7. GitHub Documentation – Repository and project management.
8. Weather API Provider Documentation – API authentication and weather data.

FINAL PROJECT SUMMARY

AI-WeatherWise API is a practical educational project that combines web development, API architecture, weather information, and recommendation logic.

The project provides a simple foundation for future development. With real-time weather API integration, advanced AI, forecasting, location services, alerts, and mobile support, the system can be extended into a complete weather intelligence platform.